Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 Reactive Subflow Spraying (RSS), an end-host load balancer that routes a small number of ordered subflows and periodically moves the worst-performing subflow using end-host congestion signals.
- C_2 Precise Fast Loss Detection (PFLD), a receiver-assisted recovery mechanism based on per-subflow sequence numbers, selective negative acknowledgments, and proactive and tail probes.
- C_3 A low-state RSS and PFLD co-design that combines multipath load balancing with precise recovery and requires no switch modification.
- C_4 Evaluation using packet-level simulation, microbenchmarks, AI collectives, LLaMA and HPC execution traces, sensitivity experiments, and a programmable-DPU prototype.

B. Computational Artifacts

The evaluator repository is https://github.com/tommasobo/SC26_OrderedChaos. The archived artifact is available at https://doi.org/10.5281/zenodo.21542397. The repository provides two top-level reproduction commands. By default, generated working data stays inside the clone and final plots are collected under results/. An optional Slurm backend can place the same data on high-capacity storage.

Artifact	Contrib.	Paper elements
A_1	C_1 to C_4	htsim source, tests, experiment manifest, public-input fetcher, local and optional Slurm workflows, analysis scripts, Figures 1 and 5 to 13, Table I, Camera Ready New Image 1, and Camera Ready New Image 2.
A_2	C_2, C_4	Figure 14 plotting inputs and scripts for the DPU summary measurements.

Figures 2 to 4 are protocol and design illustrations and therefore require no computational workflow.

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains the simulator implementation of RSS, PFLD, RACK-TLP, and the routing baselines. The machine-readable manifest fixes the complete configuration for every result. Each runner requires a new output directory and records the literal command, executable hash, input hash, runtime, memory, and pass status. Packaged tables from the paper are not used as experimental input.

Expected Results

The expected result is a new plot or table with the same axes and algorithms as the corresponding paper element. In particular, RSS should remain competitive with the multipath baselines under congestion, and RSS+PFLD should reduce loss recovery delay and exposed communication time when packets are lost.

The two supplementary camera-ready results use names that remain separate from the submitted paper numbering. Camera Ready New Image 1 presents Google RPC FCT distributions as violin plots. Camera Ready New Image 2 presents Google RPC timeout-event counts.

Completed runs collect clearly named PDFs and PNGs in one result directory. The directory also contains a PASS marker and the execution identifier used to generate it.

Expected Reproduction Time (in Minutes)

For local planning, we assume a recent laptop with 8 CPU cores, 32 GiB of memory, a local SSD, and the default two workers. Under this assumption, allow 15 to 30 minutes for the short workflow, including environment provisioning and a clean build, and 24 to 48 hours for the full workflow. These laptop values are estimates and depend on processor speed and available memory.

The table below gives measured job wall times and resource requests for the optional single-node Slurm backend. Independent Slurm jobs may run concurrently, so its end-to-end time is the longest dependency chain rather than the sum of all rows. Allow about 10 minutes for setup and about 90 minutes for the full critical path, excluding queue delay and the optional LaTeX build. These values are rounded upward from the observed runs. No experiment uses a GPU. These are parallel-run timings, not laptop estimates. The default local runner uses two workers and executes the large experiment groups sequentially, so its full workflow is substantially slower.

The reviewer quick subset focuses on Figure 1B, Figures 6A/6B, Figures 7A/7B, Figure 10, Figure 5, and Table I. These compact result families exercise the simulator, loss-recovery mechanisms, plotting, and analytical model. The optional parallel workflow should be allocated about 60 minutes, excluding scheduler queue time.

Artifact Setup (incl. Inputs)

Hardware: A Linux laptop or workstation is sufficient for the default workflow. We recommend 16 GiB of memory for the short path and 32 GiB plus 100 GiB of free storage for the sequential full path. The optional Slurm backend uses one CPU-only node per task; requests range from 4 to 192 GiB according to the experiment. No multi-node allocation, GPU, or special interconnect is required by the simulator.

Result	CPU	Memory	Full wall time	Scratch	Quick mode
Build, tests, smoke	16	64 GiB	5 min	< 0.1 GiB	same
Public traces	2	4 GiB	5 min	1.8 GiB	omit
Figure 1A	8	32 GiB	5 min	2.5 GiB	omit
Figure 1B	24	128 GiB	< 1 min	< 0.1 GiB	reduced
Figure 5, Table I	1	4 GiB	1 min	< 0.1 GiB	same
Figures 6A/6B	128	64 GiB	15 min	0.4 GiB	reduced
Figures 7A/7B	128	128 GiB	75 min shared	0.4 GiB	reduced
Figure 8	16	96 GiB	30 min	4.7 GiB	omit
Figure 9	8	64 GiB	5 min	< 0.1 GiB	omit
Figure 10	32	128 GiB	60 min	0.3 GiB	reduced
Figure 11	48	192 GiB	5 min	2.7 GiB	omit
Figure 12	48	128 GiB	30 min shared	21 GiB	omit
Figures 13A/13B	64	128 GiB	15 min	0.1 GiB	reduced
Figure 14	1	4 GiB	< 1 min	< 0.1 GiB	omit
Camera Ready New Images 1/2	12	128 GiB	15 min shared	3.1 GiB shared	omit

TABLE I

Evaluator resource guide for the optional Slurm backend. CPU is requested cpus-per-task. All jobs record wall time, peak memory, literal commands, and output provenance in the selected work directory. Times are conservative planning values rounded up from observed parallel runs. They should not be used to estimate the sequential two-worker laptop workflow. Under the laptop assumption above, budget 15 to 30 minutes for the short run and 24 to 48 hours for the full run. Simulator performance depends on processor model and effective CPU allocation, so Figure 10 and the micro-collective jobs can vary substantially across systems.

Software: The workflow supports Linux on x86-64 and 64-bit Arm. The current end-to-end test used 64-bit Arm. The required software and tested versions are:

- OrderedChaos htsim, GCC/G++ 12.3.0, and CMake 3.28.3. htsim requires C++17 and CMake 3.16 or newer.
- Python 3.13.5 and uv 0.9.24. The required package pins are NumPy 2.3.4, pandas 2.3.3, Matplotlib 3.10.8, Seaborn 0.13.2, SciPy 1.16.3, and PyMuPDF 1.27.1.
- Git 2.43.0, GNU Bash 4.4.23, GNU time 1.9, GNU coreutils 8.32, and curl 8.6.0.
- Optional parallel execution uses Slurm 25.05.4. No scheduler is needed for the default workflow.

Exact Python pins are in artifact/requirements.txt. Tectonic 0.15.0 is needed only to rebuild this appendix; the compiled PDF is included.

Datasets / Inputs: Synthetic incast, tornado, ring, and all-to-all matrices and all network topologies are versioned in the repository. LLaMA, ICON, LAMMPS, and MILC binary traces are fetched from the public SPCL trace repository at http://storage2.spcl.ethz.ch/traces/. The fetch job records and verifies every SHA-256 checksum. The measured download size is 1.8 GiB. If the public server is temporarily unavailable, the workflow records the failed input, skips only its dependent trace figure, and continues all independent experiments. The result directory then contains Figure_11_SKIPPED.txt or Figure_12_SKIPPED.txt with the reason. Re-running the full command when the server is available produces the omitted figure. Figure 14 uses three small, versioned CSV files containing the recorded DPU summary values.

Installation and Deployment: Clone the repository and run reproduce.sh in either short or full mode. The default local helper installs the specified uv, Python, and package versions under .orderedchaos-work/, builds the simulator, runs all 43 tests and the smoke simulation, and then runs each required experiment with laptop-safe parallelism. The optional –slurm backend accepts account and partition values at launch time through environment variables; no site-specific value is stored in the repository.

Artifact Execution

The computational dependency graph is $T_1 \rightarrow (T_3, T_4) \rightarrow T_5$ and $T_2 \rightarrow T_4$, where:

- T_1 Cleanly compile htsim and run all tests and the smoke case.
- T_2 Fetch and checksum public LLaMA and HPC inputs. This can overlap with T_1.
- T_3 Regenerate Figure 5, Table I, and Figure 14 from versioned inputs.
- T_4 Run independent fresh simulator matrices. Every large matrix is split by result and may execute concurrently after T_1 and, for trace jobs, T_2.
- T_5 Parse raw logs, calculate medians and percentiles, create plots, and build the visual result index.

The full and short configurations, including repetitions and random seeds, are defined in the manifest. Figure 1B pools matched loss events and uses a seed-stratified bootstrap interval. Error bars elsewhere are generated from the repeated runs and are never synthesized.

Artifact Analysis (incl. Outputs)

Raw simulator logs, command files, timing records, and metrics are written to .orderedchaos-work/orderedchaos_ae by default. The final PDFs, PNGs, and tables are collected under results/. A completed matrix has a run_status.csv whose required rows all read pass. Plotting commands consume the fresh job-specific root recorded in provenance.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 reconstructs the Figure 14 presentation from explicit DPU summary measurements and supports the probe-overhead and probe-FCT portions of C_2 and C_4.

Expected Results

The expected summary reports a maximum probe overhead of 6.352%, a 1-MiB all-probe speedup of 2.483×, retransmissions equal to corruption drops, and zero all-probe timeouts. The workflow validates the CSV schemas and creates both PDF and PNG outputs.

Expected Reproduction Time (in Minutes)

Setup is shared with A_1 and adds no extra time. Schema validation, metric calculation, plot rendering, and JSON analysis together take less than 1 minute. The complete A_2 workflow uses one CPU and less than 128 MiB of memory.

Artifact Setup (incl. Inputs)

Hardware: No special hardware is needed to evaluate the supplied summary and plotting workflow.

Software: The same pinned Python environment as A_1 is used.

Datasets / Inputs: Inputs are artifact/data/figure14_probe_overhead.csv, artifact/data/figure14_probe_fct.csv, and artifact/data/figure14_probe_correctness.csv.

Installation and Deployment: No deployment step beyond the Python environment is required.

Artifact Execution

The script validates the three inputs, computes the headline quantities, and renders both Figure 14 panels.

Artifact Analysis (incl. Outputs)

The output directory contains PDF, PNG, and JSON files. The JSON provides the numeric checks used by the evaluator.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

The commands below run directly on a Linux laptop or workstation. They require no scheduler, account name, GPU, or external storage setting.

```
git clone \
https://github.com/tommasobo/SC26_OrderedChaos
cd SC26_OrderedChaos
```

Artifact Execution

For the fastest evaluator path, run:

```
./reproduce.sh short
```

This runs fresh reduced versions of Figure 1B, Figure 6, Figures 7A/7B, and Figure 10, and regenerates Figure 5 and Table I. The command prints the final result directory. Inspect its PASS marker and clearly named plots. The quick path checks functionality and the main trends. The full workflow is used for final numerical assessment.

Run the complete workflow with:

```
./reproduce.sh full
```

The local wrapper executes the complete workflow in dependency order. Every stage creates a unique run-specific root and refuses cached experiment output. The machine-readable manifest records the complete scientific configuration. Set ORDEREDCHAOS_JOBS to adjust local worker concurrency; this changes wall time but not the scientific configuration.

For higher concurrency, a Slurm installation can execute independent stages concurrently and is recommended for the complete workflow:

```
export SCRATCH=/path/to/high-capacity/storage
./reproduce.sh short --slurm
./reproduce.sh full --slurm
```

If required by the site, supply account and partition arguments at launch through the optional environment variables documented in the repository README.

Artifact Analysis (incl. Outputs)

For each result, verify the following:

1) Every required row in run_status.csv is pass.
2) The recorded seed count and task count agree with the manifest.
3) The plot provenance names the new job-specific raw root.
4) The PDF or PNG contains the expected algorithms, trim modes, and axes.
5) The generated table reports the expected qualitative trend and its measured median or percentile interval.

Raw output is under .orderedchaos-work/orderedchaos_ae/raw; intermediate summaries are under the adjacent analysis directories; and the clearly named paper plots are under results/. If a public binary trace is unavailable, inspect SKIPPED_RESULTS.txt and the corresponding per-figure skip record; all other required status rows and plots remain independently verifiable. To clean a campaign, preserve the desired final result directory and remove .orderedchaos-work/. Optional Slurm runs place the same structure below $SCRATCH.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Activate the same Python environment used for A_1. No DPU allocation is required for the supplied Figure 14 summary workflow.

Artifact Execution

Run the plotter with a new output directory:

```
.orderedchaos-work/orderedchaos_ae/venv/bin/python \
  artifact/scripts/plot_figure14_summary.py \
  --output results/figure14-check
```

Artifact Analysis (incl. Outputs)

Expected output is PDF and PNG plus JSON reporting 6.352% maximum overhead, 2.483× speedup at 1 MiB, retransmissions equal to drops, and zero all-probe timeouts.